

Module-03 Project

Thick Server, Thin Client — REST API — Node + Express + Mock DB

Overview

This project marks a shift from client-side to backend-heavy development. You will build an **authoritative, server-first application** where the primary computation, validation, and data management are handled by **Node.js**. You will design and expose a **REST API** with **Express**, and persist data to the **file-backed JSON DB** you will build in Part 1.

The client for this application will be explicitly **thin**. Its sole responsibility is to issue requests to your API and render the results. All application logic will reside on the server. To manage state for concurrent users, **lightweight session management** is required.

Connecting to the Reference Architecture (The MUD Example)

The provided multi-player MUD codebase is the primary reference for this project. It is a complete, working example of the "Thick Server, Thin Client" architecture you are required to build. Use the example MUD codebase to **analyze its design patterns** and apply them to your own unique application.

- **The Authoritative Core** (This is the "thick" part. It contains all the app's rules and state, and is completely decoupled from the web.
- **The API Layer** (This is the translator. It wraps the core logic in an Express API, handling HTTP requests and managing player sessions.
- **The Thin Client** (This is the passive view. It only sends commands and renders the server's response.

Your task is to build a new application that follows this exact pattern.

Learning Objectives

- **Design a layered server architecture** (DB → Services → API → Client) and apply HTTP semantics.
- **Implement a server-side JSON database** for persistence using Node.js-based, file-based operations.
- **Expose a REST API (CRUD) via Express** with correct HTTP semantics and centralized error handling.
- **Centralize all business logic**, including validation and data derivations, on the server, enforcing the "thick server" model.
- **Implement session middleware** to identify and manage state for multiple, concurrent clients.
- **Build a thin client** (either a user-facing UI or a developer-facing API portal) that consumes your REST API.
- **Incorporate essential software practices** including basic testing, logging, and clear documentation.

Project Scope and Complexity

This project is a significant design challenge. Your application must manage at least **two or three related data collections** in the database you build. A simple, single-collection application is not sufficient. The goal is to model the relationships between different types of data, as this is where a thick server's logic truly shines.

- **Good Examples:** A MUD (players, rooms, items), a collaborative planner (users, projects, tasks), or a turn-based card game (games, players, decks).

Tracks & Project Archetypes

You must pick **one** of the following two tracks for your project. The core backend architectural requirements (layered server, session management, NodeJS DB) are **identical for both tracks**. However, the **design philosophy of your REST API** will be fundamentally different based on your choice.

The archetypes below describe the server's role for each track.

Choose one track and state it in both `README` and `/docs/pitch.md` files

Track 1: API Portal (Public API)

In this track, the **API itself is the product**. You are designing a general-purpose, reusable utility intended to be consumed by other developers for their own applications. The API must be logical, robust, and well-documented because its "client" is another developer's code. For an example of such a service, see the **Deck of Cards API**. (e.g. <https://deckofcardsapi.com/>)

- **API Design Philosophy:** Your API should be **decoupled and general**. It should not make assumptions about how it will be used. Think: "How would I design this service so any developer could build something cool with it?"
- **Your Deliverables:** Since the API is the product, your "thin client" is a **promotional and functional portal** for that product. It must include:
 - A documentation site that clearly explains the API's contract.
 - A live **"Try it" console** for each endpoint to allow developers to experiment.
 - Clear examples of methods, routes, parameters, request/response bodies, error shapes, and session usage.

Project Archetypes for Track 1 (Public APIs)

- **Content Generation Services**
 - **Server's Role:** To act as a specialized "expert system" that generates structured, on-demand content for a narrow and highly specific domain.
 - *e.g., The Deck of Cards API generates a new, shuffled deck on request. Similarly, your API could generate procedural text (names, descriptions), mock user data for testing, or unique IDs.*
- **Interactive State Engines (Opinionated)**
 - **Server's Role:** To act as an authoritative referee for a multi-step, **domain-specific** process, managing the state of a known resource as it is mutated by a sequence of requests. Its purpose is to be a **reusable building block**, not a complete application backend.
 - *e.g., The Deck of Cards API manages the state of a deck (a known resource) across multiple calls. Your API could similarly provide a reusable engine for a turn-based system, a workflow tracker, or an ordered mutable list.*
- **Protocol & Sequencing Engines**
 - **Server's Role:** To act as an un-opinionated public utility for managing the **protocol of an interaction**. Instead of managing domain-specific data (like cards or products), its job is to manage the *order, sequence, and priority* of requests, providing a general-purpose mechanism that other applications can use.
 - *e.g., An API that provides a generic turn-taking service for multiple clients, a simple resource locking mechanism to prevent conflicts, or a priority queue for ordering asynchronous tasks.*

- **Computational Engines & Analysis**
 - **Server's Role:** To perform resource-intensive calculations, run complex models, or analyze large datasets that are impractical for a client to handle. The API exposes the *results* of this heavy lifting. Alternatively, the results might be based on proprietary private data.
 - *e.g., An API that provides scientific model forecasts, statistical analysis on a large dataset, or **real-time observational data** (like weather or transit information).*
- **Authoritative Knowledge Bases**
 - **Server's Role:** To act as the "single source of truth" for a rich, interconnected dataset, serving queries about the relationships between entities.
 - *e.g., An API that serves as a definitive reference for an interconnected domain, like a catalog of recipes and their ingredients, or a library of books and authors.*

Track 2: Thin UI (Private API)

In this track, the **interactive, multi-user experience is the product**. The API is a private, internal mechanism designed for one specific purpose: to make *your* application's front-end work. It is not intended for general consumption. The server acts as the central authority that manages a shared state between multiple, concurrent clients, similar to the **MUD example**.

- **API Design Philosophy:** Your API can be **specific and tightly-coupled** to the needs of your UI. It can have specialized endpoints that perform complex, application-specific actions because you control the only client that will ever use it. Think: "What is the most efficient endpoint I can create to power this specific feature?"
- **Your Deliverable:** Your "thin client" is the **view and interface for your product**. It must be a minimal web application that:
 - Accomplishes a concrete, multi-user task.
 - Remains **lean** (view logic and data requests only), with all application rules and logic handled by the server.
 - Requires sessions for protected routes to manage the state of each connected user.
 - Persist data via the mock NodeJS DB.

Project Archetypes for Track 2 (Private APIs)

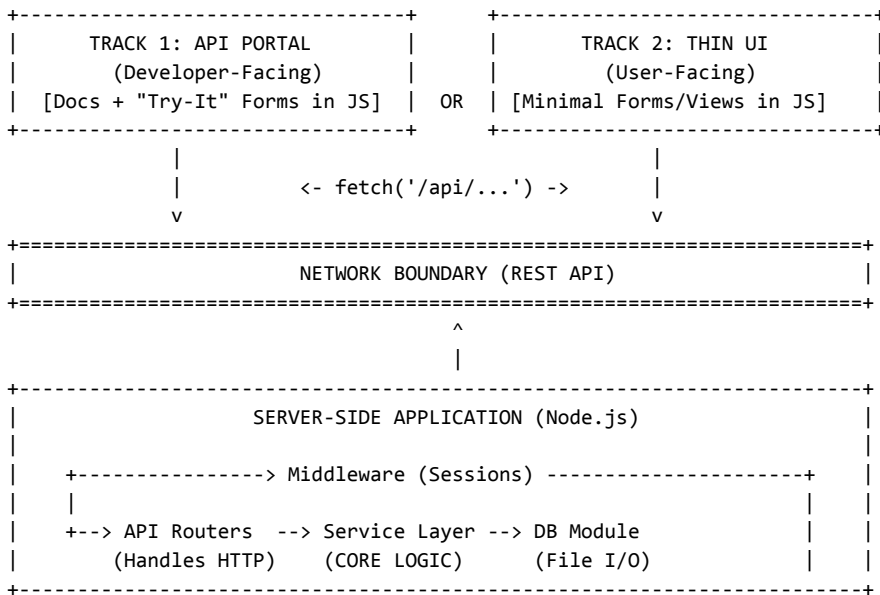
- **Archetype: Collaborative Workspaces**
 - **Server's Role:** To be the single source of truth that allows multiple users to work **cooperatively** on a shared resource.
 - *e.g., Think about a shared to-do list, a collaborative story-writing app, a simple project planning board, or a group reservation system.*
- **Archetype: Competitive Arenas**
 - **Server's Role:** To act as an impartial referee for a game or competition, enforcing rules between users who are working **adversarially**.
 - *e.g.: Consider any turn-based multiplayer game: a simple board game, a card game, a trivia competition, or a word game*
- **Archetype: Persistent Shared Worlds**
 - **Server's Role:** To manage a persistent, dynamic environment where users can **co-exist** and interact with the world and each other.
 - *e.g. Think about a simple MUD-like environment or a real-time chat room with different "channels."*

Project Requirements

1. Server-side NodeJS Database (Refactor thecode from the Browser Database Lab)
2. REST API Endpoints (Your own Design) that also uses your version of the NodeJS DB
3. Thin Client (Connects and uses your server, REST API, and server database)

Architectural Blueprint: A Unified Server for Any Client

A key goal is to build an API-first server that is independent of its client. The diagram below illustrates how both tracks connect to the same server application. Use this as your structural blueprint.



Guidance: Structuring Your Server-Side Application

Your **Server-Side Application** is the entire engine; the **REST API** is the control panel it exposes. We will organize the engine using a layered folder structure based on **Separation of Concerns**.

Recommended Folder Structure

```

your-project-root/
├── data/           # Your JSON database files
├── public/         # Your thin client (HTML, CSS, client-side JS)
├── src/            # ALL your server-side source code
│   ├── api/       # API Layer: Express routers, handles req/res. NO business logic.
│   ├── services/  # Service Layer: The BRAIN of your app. All rules and logic live here.
│   ├── db/        # Data Layer: Your DB from Part 1. Handles file reading/writing.
│   └── middleware/ # Session management, error handling.
├── server.js      # Entry point: creates the Express app and starts it.
└── package.json   # Metadata & Configuration information for Building & Running NodeJS App

```

Project Breakdown by Part

Part 1: Server-Side Node.js Database

- Build a file-backed data store using the Node.js `fs` module.
- **Requirements:** One JSON file per collection, atomic writes (`.tmp` → `rename`), and an in-memory cache.

Part 2: REST API Endpoints (Express)

- Build the brain of your application.
- **Requirements:** Design and implement RESTful endpoints, use Express routers, implement session management, and create a centralized JSON error handler.

Part 3: Thin Client Implementation

- Build the client based on your chosen track.
- **Requirements:** Use `fetch` with `async/await`. The client must contain no business logic. For a UI, show clear feedback states. For a portal, provide clean documentation and working "try-it" forms.

Architectural Expectations

This project is a **thick-server application**. The server (Node.js) owns all the application logic. The client is **thin**: it issues HTTP requests and renders responses. Your code must be organized into modules with a clear separation of concerns.

Minimum Architectural Expectations

- **Layered Modules:** Your server-side code in the `src/` directory must be organized into at least **four distinct layers**:
 - `src/db/` (The Persistence Layer)
 - `src/services/` (The Domain Logic Layer - this is the "brain")
 - `src/api/` (The API/Routing Layer - handles Express routes)
 - `src/middleware/` (The Middleware Layer - for sessions, logging, etc.)
- **API Endpoint Complexity:** Your API must be sufficiently complex to manage your data model. It must include:
 - **Full CRUD Support:** A complete set of RESTful endpoints (`GET` list, `GET` by ID, `POST`, `PUT/PATCH`, `DELETE`) for at least **one primary resource** (e.g., `players`, `projects`).
 - **At least one "Relational" Endpoint:** An endpoint that manages the relationship between two resources (e.g., `GET /api/users/:userId/tasks` to get tasks for a user, or `POST /api/projects/:projectId/members` to add a member to a project).
- **Minimum Server Stack:** Your server application must be built with and integrate the following core technologies:
 - **Runtime & Package Manager:** Node.js and `npm`.
 - **Web Framework:** **Express**, used for all routing and request/response handling.
 - **Middleware Integration:** Your Express app must integrate middleware for:
 - **Request Body Parsing:** `express.json()`.
 - **Session Management:** A custom or library-based middleware to manage session IDs.
 - **Centralized Error Handling:** A custom, final middleware to catch all errors and produce a consistent JSON error response.
 - **Persistence:** The built-in Node.js `fs` module for all file I/O in your database layer.

Server-Side Integration Choices

Your server must meet the minimum stack requirements. To add satisfactory complexity and demonstrate a deeper understanding of backend architecture, select and integrate **at least one** of the following professional patterns.

A) Advanced Session Management (Eligible for Both Tracks)

The baseline requirement is a simple, custom session middleware. This option allows you to implement a more robust, industry-standard session strategy.

- **Choice 1: Server-Managed Sessions (Stateful)**
 - **Eligible:** `express-session` library.
 - **Use When:** You want a battle-tested solution for managing sessions on the server. The server "remembers" each client and can easily invalidate a session if needed.
 - **Expectations:** `express-session` is correctly configured with the in-memory store. Session data is created on a user's first request and is available in `req.session` on all subsequent requests.
- **Choice 2: Client-Managed Sessions (Stateless with JWT)**
 - **Eligible:** JSON Web Tokens (using a library like `jsonwebtoken`).
 - **Use When:** You want to build a truly stateless API where the server doesn't need to store session data. The client holds an encrypted token that proves who they are.
 - **Expectations:** On login/join, the server signs a JWT containing user/session info and sends it to the client. A middleware on protected routes validates the token from the `Authorization` header on every request.

B) API Specification & Auto-Generated Docs (Primarily for Track 1)

The baseline is a manually written `api_endpoints.md` file. This option is the professional workflow for designing and documenting a public-facing API.

- **Eligible:** OpenAPI Specification (formerly Swagger) with `swagger-ui-express`.
- **Use When:** You are on **Track 1** and want to create a professional, interactive API portal. This is the industry-standard method for designing and documenting REST APIs that are meant to be consumed by other developers.
- **Expectations:** You create an `openapi.yaml` (or `.json`) file that formally defines your API's resources and endpoints. You use the `swagger-ui-express` library to serve a live, interactive documentation portal (e.g., at `/api-docs`) that is automatically generated from your specification file.

C) Real-Time Updates with Server-Sent Events (SSE) (Primarily for Track 2)

The baseline is having the client repeatedly poll the server for updates. This option implements a more efficient, "push-based" architecture for a truly real-time user experience.

- **Eligible:** Server-Sent Events (using a library like `sse-express`).
- **Use When:** You are on **Track 2** and want to provide **instant updates** to all connected clients without forcing them to constantly poll your server. This is the modern architectural pattern for live games, chat apps, and collaborative tools.
- **Expectations:** You create a special `/events` endpoint on your server that holds open a connection to the client. When one user performs an action that changes the shared state (via a standard `POST` request), your server handler then **pushes** an event through the open SSE connection to all other relevant clients. Your front-end client code listens for these events and updates the UI instantly, without needing a `setInterval` polling loop.

API Design Caveats & Best Practices

As you design and build your REST API, keep these professional principles in mind. They will help you create a service that is clean, scalable, and easy to use.

- **The Server is the Single Source of Truth**
 - The client is a "dumb" terminal. It should never contain hidden state or business logic. If the client needs to know something, it must ask the server. When the client sends data, the server must re-validate it, because the client cannot be trusted.
 - **Serve the Client from the Same Origin to Avoid CORS**
 - Cross-Origin Resource Sharing (CORS) is a common source of complexity and errors in web development. The simplest way to avoid it entirely is to serve your thin client's static files (HTML, CSS, JS) directly from your Express server. Unless you have a specific reason to host your client on a different domain, this is the recommended approach.
 - **Use Query Parameters for Filtering, not Path Parameters**
 - A URL path should identify a *resource* (e.g., `/api/users/123`).
 - URL query parameters should be used to *filter, sort, or paginate* a collection of resources (e.g., `/api/users?status=active&sort=name`).
 - **Plan for Pagination**
 - While not a strict requirement for this project, professional APIs that return lists of items are always paginated to avoid sending massive amounts of data. A common pattern is to use `?limit=` and `?offset=` query parameters and to return the total number of items in a custom HTTP header like `X-Total-Count`.
 - **Strive for Statelessness**
 - The core principle of REST is that every request from a client should contain all the information the server needs to fulfill it. Your session management system helps with this: the session ID sent with each request allows the server to load the necessary context without having to "remember" the client from a previous, unrelated request.
-

Thin Client Responsibilities

While the server is the "brain" of your application, the thin client still has a role: to provide a clean, usable interface for the user to interact with the API. A good thin client is not about complex logic; it is about clearly communicating the state of the network interaction.

- **Show Clear Feedback States (Core Responsibility)**
 - This is the most important job of your thin client. Because every action involves a network request, you must visually communicate the state of that request to the user.
 - **Loading:** Show a loading indicator (e.g., a spinner, a "Loading..." message) and disable buttons while a `fetch` request is in progress to prevent duplicate submissions.
 - **Success:** Show a clear confirmation when an action succeeds.
 - **Error:** If the API returns an error (like a 400 or 500), display a user-friendly error message.
 - **Empty:** If a `GET` request for a list returns an empty array, display a helpful message instead of a blank screen.
 - **Keep Logic Thin**
 - The client should contain no business logic, derivations, or complex validations. Its only job is to capture user input, send it to the server, and render the server's authoritative response.
 - **Simple and Readable Layout**
 - Your client does not need a complex responsive design. It should be designed for a standard laptop viewport (e.g., 1280px wide) and be clean, simple, and readable.
-

Acceptance Criteria (Definition of Done)

These criteria define the specific, testable outcomes required to consider each part of the project complete.

Part 1: Node.js Database

The goal is to prove your database module is a reliable, persistent data store that can be used by the rest of your application.

- **Automated Test:** A provided test script (`/tests/db-test.js`) can be run against your database module in isolation (e.g., with `node tests/db-test.js`), and **all tests must pass**.
- **Persistence:** The test script will specifically verify that data **persists across DB re-initializations**. It will create an item, "shut down" the DB, "restart" it, and confirm the item can still be read.
- **Atomic Writes:** Your code must implement the `.tmp -> rename` strategy. You can demonstrate this by explaining your implementation in the comments of your DB module's `save` or `write` function.

Part 2: REST API

The goal is to prove your API is a well-behaved, stateful, and secure interface. You will use an API client like **Postman, Insomnia, or** to verify this.

- **Endpoint Functionality:** All designed CRUD endpoints for at least one resource must be functional.
 - `POST` requests with valid data return `201 Created` and the created resource.
 - `GET` requests return `200 OK` and the correct data.
 - Requests for a non-existent resource ID return a `404 Not Found`.
 - Requests with invalid or missing data return a `400 Bad Request` along with your standard JSON error envelope.
- **Session Management:** The session lifecycle must be observable:
 - The very first request made to any protected endpoint from a new client must receive a **header** in the response.
 - All subsequent requests from that client must automatically include the `Cookie` header.
- **Traceability:** A `GET /health` endpoint must exist and respond with a JSON object that includes the current session ID (e.g., `{ "ok": true, "sessionId": "..." }`). This proves the server is correctly identifying the client on each request.

Part 3: Thin Client

The goal is to prove that your client correctly uses the API and maintains no business logic.

- **Demonstrable End-to-End Flow:** You must provide a **screen recording (GIF or short video)** that clearly demonstrates one complete CRUD cycle. For example, a user creates an item, sees it appear in a list, updates it, sees the change, and then deletes it, watching it disappear.
 - **Visual Feedback:** The recording must clearly show visual feedback states for the user, such as a "loading..." indicator during API calls, success messages (e.g., "Item created!"), and user-friendly error messages when the API returns an error.
 - **Updated Documentation:** The `README.md` file must be updated with a screenshot or the GIF of the working client. It must also include `curl` or `fetch` code snippets for at least two of your main API endpoints as usage examples.
-

Documentation

Treat docs like part of the product. Your repo should have a **public-facing README.md** that reads like an app landing page (what it is, how to use it, how it works, demo links) and a **developer docs area** in `/docs/` that contains all planning and course artifacts (pitch, roadmap, architecture sketch, API endpoint details, and per-sprint definitions of done). This split keeps the repository professional for the public between users and developers.

Public README (README.md)

Use this as a landing page for users and reviewers (i.e. no “homework” wording).

Recommended order

1. **Product Name & One-Line Summary:** A short value statement: what your application or service is and why it exists.
2. **Screenshots / Demo GIF:** One or two screenshots, or a short GIF from `docs/media/`, showing your client in action.
3. **How to Run:** Clear, simple, and bulletproof instructions for running the server locally (e.g., `npm install`, then `npm start`). State any requirements (e.g., "Requires Node.js v18+").
4. **How It Works (High-Level):**
 - **Architecture in Brief:** A one-sentence description of your thick-server, thin-client architecture.
 - **Technology Stack:** Briefly list the key technologies used (Node.js, Express, etc.).
5. **API Usage Examples (High-Level):**
 - Provide two or three tiny `curl` or `fetch` examples for your most important endpoints. Keep these examples small and link to the full details in your developer docs.
6. **Developer Docs (Links):** A short list linking to the detailed planning and design documents inside the `/docs/` directory.

Development Docs (/docs/)

All planning artifacts and course deliverables live here.

Suggested layout

```
docs/  
  pitch.md  
  roadmap.md  
  architecture_sketch.md  
  api_endpoints.md  
  dod-sprint1.md  
  dod-sprint2.md  
  dod-sprint3.md  
  dod-sprint4.md  
  media/  
    demo.gif  
    screenshot.png
```

File templates (copy/paste)

docs/pitch.md

```
# Pitch  
**Track:** API Portal / Thin UI  
**Product (one line):** What it is and why it exists.  
**Problem & User:** Who it helps and the need.  
**Core Data Entities (2-3 sentences):** Describe the main data collections you will manage (e.g., "This app will manage  
Players, Rooms, and Items...").
```

docs/roadmap.md

```
# Roadmap  
  
## MVP (Sprint 3)  
* Feature A for Resource 1...  
* Feature B for Resource 1...  
* Vertical Slice: A full CRUD flow for one resource is working end-to-end.  
  
## Full Features (Sprint 4)  
* CRUD support for Resource 2...  
* Relational endpoint between Resource 1 and 2...  
* Advanced integration choice (e.g., OpenAPI, SSE)...
```

docs/architecture_sketch.md

```
# Architecture Sketch  
Add a simple diagram (ASCII or an image) showing the relationship between your client, API layer, service layer, & DB.  
* **Module Map:** List your key `src/` modules and their responsibilities.
```

docs/api_endpoints.md

```
# API Endpoint Reference
For each of your major endpoints, provide the following:

### `GET /api/resource`
* **Description:** Retrieves a list of all resources.
* **Example Request:** `curl http://localhost:3000/api/resource`
* **Example Response (200 OK):**
  ``json
  [ { "id": "...", ... }, ... ]
  ``

---
### `POST /api/resource`
* **Description:** Creates a new resource.
* **Example Request Body:**
  ``json
  { "name": "...", "value": "..." }
  ``
* **Example Response (201 Created):**
  ``json
  { "id": "...", "name": "...", "value": "..." }
  ``
```

Tone & review checklist

- **README** reads like a product page (features, demo, how it works, how to run).
- **docs/** holds all the planning and design artifacts.
- No secrets in the repo; endpoints shown with small examples.
- Screenshots/GIFs stored under **docs/media/** and linked where needed.

Project Management & Definition of Done

This project requires you to manage your work using professional tools and practices. You will use **both** a GitHub Project board and per-sprint Definition of Done (DoD) files.

- **GitHub Project Board:** This is your **working task list**. Before starting Sprint 3 (MVP), you must create a project board and populate it with issues representing the individual tasks for that sprint (e.g., "Create GET /api/players endpoint," "Build player creation form"). This is where you will track your day-to-day progress.
- **Definition of Done Files:** These are your **sprint contracts**. A `dod-sprintX.md` file is a high-level checklist that defines the overall goals for a sprint. Before submitting a sprint for grading, you should be able to check off every item in the corresponding DoD file.

docs/dod-sprint1.md

```
# Sprint 1 – Definition of Done (Database)
- [ ] DB module is refactored for Node.js with atomic writes.
- [ ] All provided test scripts for the database module pass.
- [ ] The `docs/` directory has been created in the repository.
```

docs/dod-sprint2.md

```
# Sprint 2 – Definition of Done (Planning)
- [ ] `pitch.md` is complete and committed.
- [ ] `roadmap.md` (MVP vs. Full) is complete and committed.
- [ ] `architecture_sketch.md` is complete and committed.
- [ ] `api_endpoints.md` for the MVP is complete and committed.
- [ ] A GitHub Project board has been created and populated with issues for Sprint 3.
- [ ] A link to the project board has been added to the main `README.md`.
```

docs/dod-sprint3.md

```
# Sprint 3 – Definition of Done (MVP Vertical Slice)
- [ ] The API for at least one resource is fully functional (CRUD).
- [ ] Session management is working and can be demonstrated.
- [ ] The thin client can perform a full CRUD operation against the API.
- [ ] Evidence: The Sprint 3 project board filter link shows all MVP tasks as "Done."
```

docs/dod-sprint4.md

```
# Sprint 4 – Definition of Done (Full Feature & Polish)
- [ ] All features from the roadmap are complete, including relational endpoints.
- [ ] The chosen "Server-Side Integration Choice" (e.g., OpenAPI, SSE) is implemented and working.
- [ ] The final `README.md` is professional and complete.
- [ ] A demo video or GIF is created, stored in `docs/media/`, and linked in the `README.md`.
- [ ] Evidence: The Sprint 4 project board filter link shows all tasks as "Done."
```

Project Timeline & Roadmap (Sprints)

Sprint	Days	Focus
Sprint 1	Days 1-3	Database Implementation
Sprint 2	Days 4-6	Pitch & Planning
Sprint 3	Days 7-14	MVP Vertical Slice
Sprint 4	Days 15-21	Full Feature & Polish

Grading Rubric (Total 100 pts)

This project will be graded across four distinct sprints, each with its own deliverables and point value.

Sprint 1 — NodeJS Server Database (20 pts)

- **Refactor Browser Database (10 pts):** The browser database lab is successfully refactored to run as a local Node.js module, using the `fs` module for persistence with atomic writes.
- **Pass Provided Test Scripts (10 pts):** The database module passes all provided test scripts, proving that it is a reliable and persistent data store.

Sprint 2 — Pitch & API Design (15 pts)

- **Pitch & Roadmap (5 pts):** The `/docs/` directory contains a clear project pitch and a roadmap that distinguishes between MVP and full features.
- **API & Architecture Design (10 pts):** The `/docs/` directory contains a logical API endpoint design and a clear architecture sketch for the full server application.

Sprint 3 — MVP Vertical Slice (35 pts)

- **REST API Execution (20 pts):** A vertical slice of the API (for one resource) is functional, using the database from Sprint 1. It demonstrates correct HTTP semantics, error handling, and working session management.
- **Thin Client MVP (15 pts):** The thin client can perform a full, end-to-end CRUD flow for the single resource implemented in the MVP API, demonstrating a clear separation of concerns.

Sprint 4 — Feature-Complete & Polish (30 pts)

- **Full Feature Completion (15 pts):** All planned API features from the Sprint 2 roadmap are delivered, correctly handling the relationships between multiple data collections.
- **Client Polish & Final README (15 pts):** The chosen client track is well-executed and polished. The final `README.md` is professional, comprehensive, and includes a demo GIF and API usage examples.

Bonus — Showcase Bonus (+10 pts)

- Outstanding projects that demonstrate exceptional design, execution, and documentation may be eligible for a showcase bonus.